BGP 101: A Practical Guide to the Border Gateway Protocol

Carlos M. Martinez

2026-10-18

Version v0.1.0

Carlos M. Martinez
XT6 Labs

Version history

Version	Date

Table of contents

About this course 5
References & Further Reading . . . 5

1 BGP 101 - Basic Concepts 7
1.1 What is BGP ? . . . 7
1.2 Core ideas behind BGP . . . 7
1.3 On Sessions . . . 8

2 BGP Messages 10
2.1 BGP Finite State Machine (FSM) . . . 10
2.2 1. OPEN (Type 1) . . . 11
2.3 2. UPDATE (Type 2) . . . 12
2.4 3. NOTIFICATION (Type 3) . . . 13
2.5 4. KEEPALIVE (Type 4) [3] . . . 13
2.6 5. ROUTE-REFRESH (Type 5) [13] . . . 14

3 Path Attributes and BGP Best Path Selection 15
3.1 Path Attributes . . . 15
3.2 BGP Best Path Selection Algorithm . . . 17

4 Announcing My Routes to the World Using BGP 19
4.1 Multiprotocol BGP and Address Families . . . 19
4.2 The `network` Command . . . 19
4.3 Anchoring Routes: The “Pull-up” Route . . . 20
4.4 Historical Context: BGP Synchronization . . . 21
4.5 Redistributing IGPs into BGP . . . 21
4.6 Influencing Inbound Traffic: The MED . . . 22
4.7 Troubleshooting and Verification . . . 22
4.8 Summary Checklist . . . 23

5 Chapter 06: iBGP Mechanics and Internal Propagation 24
5.1 1. Peering on Loopbacks . . . 24
5.2 2. The iBGP Split-Horizon Rule . . . 24
5.3 3. The Next-Hop Behavior (The #1 iBGP Issue) . . . 25
5.4 4. TTL Differences . . . 26
5.5 5. Scaling iBGP: Route Reflectors . . . 26
5.6 Summary Comparison . . . 27
5.7 Troubleshooting Checklist . . . 27

6 Accepting Routes from Others 28
6.1 1. Defining Routing Policy . . . 28

6.2 2. The Toolset: Prefix-Lists, AS-PATH Lists, and Route-Maps 28
6.3 3. Hands-on Scenario 1: The "Default Route Only" Filter 29
6.4 4. Hands-on Scenario 2: Filtering by Origin (AS-PATH Regex) 30
6.5 5. Hands-on Scenario 3: Defensive Hygiene & Prefix Limits 31
6.6 6. Verification and "Soft Resets" 31

7 Traffic Engineering with BGP **33**
7.1 1. The Two Directions of Traffic 33
7.2 2. Influencing Outbound Traffic 33
7.3 3. Influencing Inbound Traffic 34
7.4 4. BGP Communities: The "Remote Control" 35
7.5 5. Summary: The TE Hierarchy 36

About this course

This course provides a comprehensive introduction to the **Border Gateway Protocol (BGP)**, the "glue" that holds the Internet together. It explores how network operators exchange routing information, enforce policies, and exert control over traffic flows to maintain a stable, loop-free global network.

Unlike theoretical deep dives, this course is designed for **practical observability and hands-on engineering**. We move beyond the "how it works" to the "how to build and fix it." By using modern, containerized environments like **FRR (Free Range Routing)** and **Containerlab**, students can simulate complex, multi-AS topologies on their own machines, gaining the confidence to manage real-world infrastructure.

Key topics covered include:

- **Core Concepts:** Understanding Autonomous Systems (ASNs), the BGP Finite State Machine (FSM), and the fundamental distinction between Internal (iBGP) and External (eBGP) sessions.
- **Protocol Mechanics & Messaging:** A deep dive into BGP message types and the underlying TCP-based transport that ensures reliability.
- **Path Selection & Policy:** Mastering the BGP Best Path Selection algorithm and understanding how Path Attributes (Weight, Local Pref, MED, AS_PATH) are used to manipulate traffic flow.
- **Internal BGP (iBGP):** Navigating the complexities of internal route propagation, the split-horizon rule, and scalable architectures using Route Reflectors.
- **Route Origination & Hygiene:** Professional techniques for announcing prefixes to the world, including the "Exact Match" rule, route anchoring with Null0/blackhole routes, and safe redistribution strategies.
- **Verification & Troubleshooting:** Developing a systematic approach to debugging BGP sessions, including interpreting the BGP table, verifying advertisements, and using soft-resets to apply policy changes without disruption.

References & Further Reading

Official Specifications

- **RFC 4271:** A Border Gateway Protocol 4 (BGP-4).
- **RFC 4456:** BGP Route Reflection: An Alternative to Full Mesh Internal BGP.
- **RFC 1997:** BGP Communities Attribute.

Tools used in this course

- **FRR (Free Range Routing):** The IP routing protocol suite for Linux and Unix platforms.
- **Containerlab:** Orchestration for container-based networking labs.

Community Resources

- **Hurricane Electric BGP Toolkit:** Essential tool for checking global routing information and ASN details.
- **NANOG BGP Tutorials:** High-quality presentations and videos from the North American Network Operators Group.
- **BGPStream:** A free resource for monitoring BGP hijacking, leaks, and outages.

1 BGP 101 - Basic Concepts

Author Carlos Martinez-Cagnazzo **Version** 0.1

1.1 What is BGP ?

The *Border Gateway Protocol* is a key element of the Internet protocol stack. It essentially holds the Internet together by providing a clean mechanism that allows network operators to exchange routing information while enforcing routing policy and exerting control over traffic flows.

The ultimate goal of BGP is to enable routers to build routing tables that can be used to route traffic over a very large network. These routing tables MUST describe a loop-free topology where paths are optimized by an algorithm (*BGP Best Path*) applying set of criteria (*attributes*).

1.2 Core ideas behind BGP

We will define a *network* in this context as a set of devices, links and IPv4/IPv6 prefixes under the *administrative control* of a single authority. Networks in BGP are identified by a 32 bit number called an *Autonomous System Number*.

BGP-speaking routers ("*bgp speakers*") establish *sessions* with other bgp speakers. We differentiate between:

- *Internal* BGP sessions (iBGP) : sessions where both ends of the session belong to the same autonomous system.
- *External* BGP sessions (eBGP) : sessions where both ends belong to different autnomous systems.

```
graph LR
    subgraph AS 65001
    R1[Router 1] -- iBGP --- R2[Router 2]
    end
    subgraph AS 65002
    R3[Router 3]
    end
    R2 -- eBGP --- R3
```

The information distribution policies for both sessions will be different. Both types of sessions are important and will play different key roles as we engineer our network. It is also common for a bgp speaker to host a mix of iBGP and eBGP sessions.

The goal of a established sessions is to exchange *messages*. Messages are the core functionality of BGP, where routing information is exchanged among peers and then used to build routing tables.

1.3 On Sessions

A session exists as a TCP connection, normally on port 179. For a session to be correctly established both ends have to be correctly configured. This means, in particular that authentication credentials haver to be correctly configured on both ends and that each end needs to know beforehand the remote as-number and this has to be in agreement.

Imagine that router A, belonging to as 1, wants to establish a session with router B which belongs to AS 2. Agreement means that router A "wants to establish a session with a router on AS2 and router B a session with AS1".

Sessions go through a lifecycle expressed as a *finite state machine (FSM)*. While we will cover the details in the **BGP Messages** chapter, the general flow is:

High-Level BGP Session Lifecycle

```
graph LR
    IDLE([IDLE]) --> CONNECT([CONNECT])
    CONNECT --> OPENSENT([OPENSENT])
    OPENSENT --> OPENCONFIRM([OPENCONFIRM])
    OPENCONFIRM --> ESTABLISHED([ESTABLISHED])

    ESTABLISHED -.->|Error| IDLE
```

As per RFC4271, the BGP Finite State Machine (FSM) manages peer relationships through six main states—Idle, Connect, Active, OpenSent, OpenConfirm, and Established—ensuring reliable TCP-based communication. It moves from initial configuration to full exchange via these steps, with Established being the final state where routing information is exchanged.

1.3.1 The Six BGP States

- Idle (Initial State): BGP detects a start event and tries to initiate a TCP connection to the peer. It refuses all incoming connections.
- Connect: BGP initiates the TCP connection. If the 3-way handshake succeeds, it moves to OpenSent. If it fails, it moves to Active.
- Active: BGP tries to initiate a TCP session. If it fails to connect, it stays in this state, often indicating a misconfiguration, such as incorrect peer IP addresses, or a network issue.
- OpenSent: The TCP connection is active, and BGP sends an OPEN message to the peer to negotiate parameters.
- OpenConfirm: BGP waits for a KEEPALIVE message to confirm the OPEN message.
- Established: The final state where BGP peers successfully exchange UPDATE, KEEPALIVE, and NOTIFICATION messages. [1, 2, 3, 4, 5, 6]

Several motives can make a session go back from ESTABLISHED to IDLE:

- TCP connection reset
- A explicit NOTIFICATION message closing the session
- HoldTimer expiration (the hold timer is reset each time a keepalive or an actual message is received)

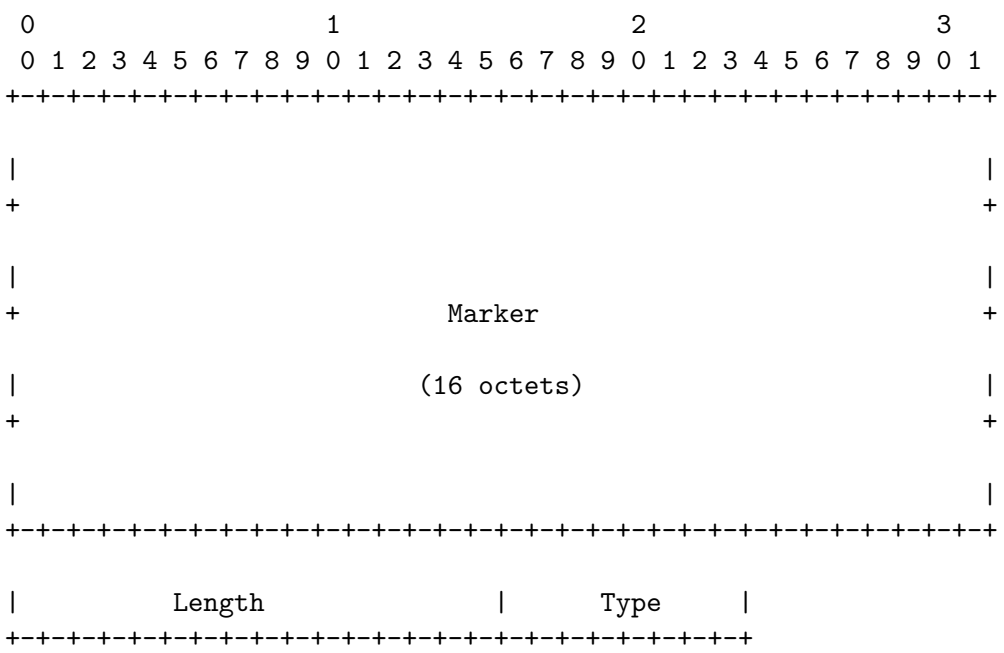
1.3.1.1 Common Issues and Transitions

- Stuck in Active: This is a common issue often caused by improper configuration or connectivity problems preventing TCP port 179 from opening.
- Idle Failure: If any error occurs, the BGP session returns to the Idle state.
- Role of TCP: As a transport-layer protocol, BGP depends on a successfully established TCP session before any BGP state transition occurs.
- Official Resource: Detailed procedures for the FSM are outlined in the RFC 1163

2 BGP Messages

BGP-4 utilizes four primary message types defined in RFC 4271 to establish and maintain peer relationships, along with a fifth common type for route management. All BGP messages share a standard 19-octet header consisting of a Marker (16 bytes), Length (2 bytes), and Type (1 byte). [1, 2, 3, 4]

```
sequenceDiagram
    participant R1 as Router A
    participant R2 as Router B
    Note over R1,R2: TCP 3-Way Handshake (Port 179)
    R1->>R2: OPEN
    R2->>R1: OPEN
    R1->>R2: KEEPALIVE
    R2->>R1: KEEPALIVE
    Note over R1,R2: Session Established
    R1->>R2: UPDATE (Prefixes + Attributes)
    R2->>R1: UPDATE (Prefixes + Attributes)
```



2.1 BGP Finite State Machine (FSM)

Before we dive into the specific message formats, it is important to understand how these messages drive the lifecycle of a BGP session. The FSM ensures both routers agree on parameters before

exchanging routes.

```
flowchart TD
    IDLE([IDLE])
    CONNECT([CONNECT])
    ACTIVE([ACTIVE])
    OPENSENT([OPENSENT])
    OPENCONFIRM([OPENCONFIRM])
    ESTABLISHED([ESTABLISHED])

    IDLE -- "Start Event" --> CONNECT
    CONNECT -- "TCP Success" --> OPENSENT
    CONNECT -- "TCP Fail" --> ACTIVE
    ACTIVE -- "TCP Success" --> OPENSENT
    ACTIVE -- "TCP Fail / Retry Timer" --> CONNECT

    OPENSENT -- "Recv OPEN / Send KEEPALIVE" --> OPENCONFIRM
    OPENCONFIRM -- "Recv KEEPALIVE" --> ESTABLISHED

    %% Error conditions
    ERROR{Error Detected?}
    OPENSENT -- "Notification/Error" --> ERROR
    OPENCONFIRM -- "Notification/Error" --> ERROR
    ESTABLISHED -- "Notification/Error" --> ERROR

    ERROR -- "Reset TCP / Close Session" --> IDLE

    style ESTABLISHED fill:#9f9,stroke:#333
    style ACTIVE fill:#f99,stroke:#333
```

2.2 1. OPEN (Type 1)

The OPEN message is the first message sent after the TCP 3-way handshake to initiate a BGP session. It includes critical negotiation parameters: [5, 6, 7, 8]

- BGP Version: Must be version 4.
- Autonomous System (AS) Number: Identifies the sender's AS.
- Hold Time: The maximum seconds allowed between messages before a session is declared dead.
- BGP Identifier (Router ID): Typically the router's highest loopback IP.
- Optional Parameters: Used to negotiate capabilities like Multiprotocol BGP (MP-BGP) or 4-byte AS numbers. [1, 2, 6, 9, 10]

2.3 2. UPDATE (Type 2)

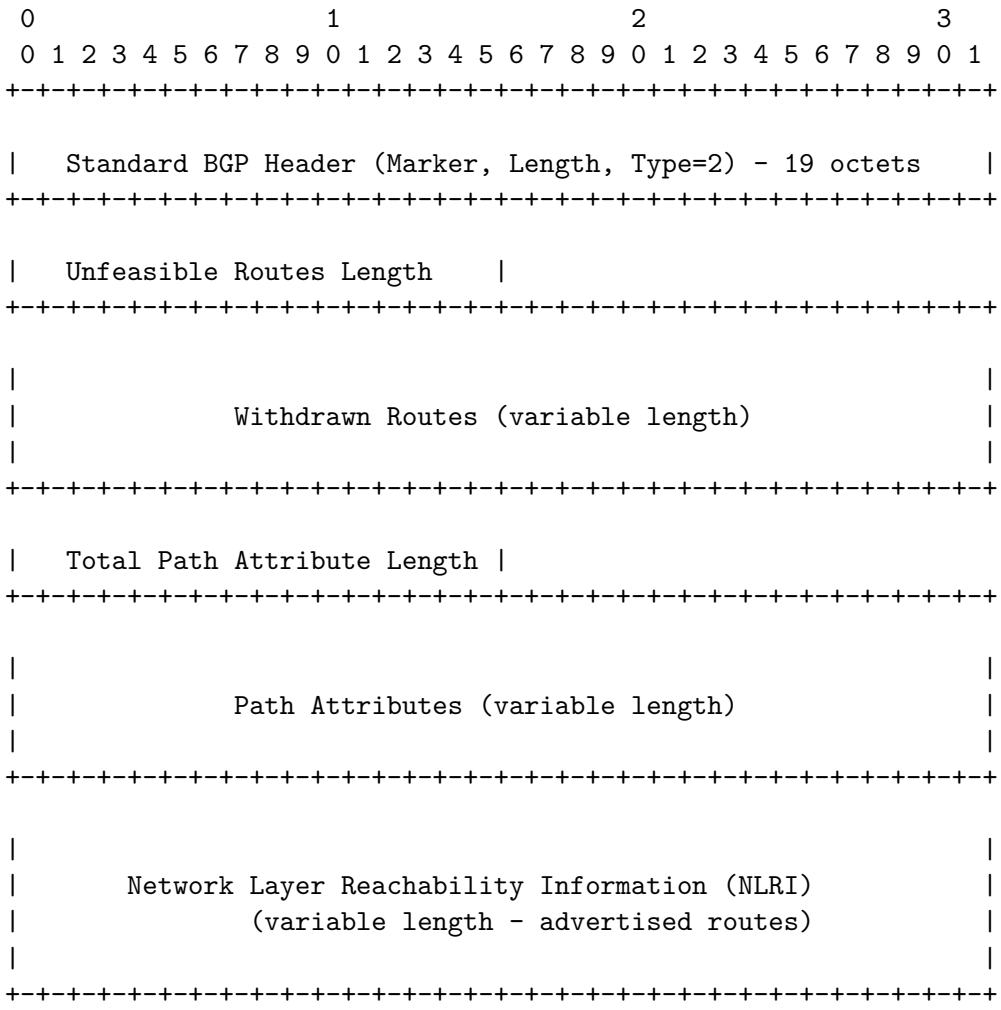
The UPDATE message is used to exchange routing information. A single message can simultaneously advertise and withdraw routes: [2, 5]

```
graph TD
    UPDATE[UPDATE Message]
 --> WITHDRAW[Withdrawn Routes]
    UPDATE --> ATTR[Path Attributes]
    UPDATE --> NLRI[NLRI - Advertised Prefixes]

    ATTR --> NH[Next Hop]
    ATTR --> AS[AS Path]
    ATTR --> LP[Local Pref]

    NLRI --> P1[192.0.2.0/24]
    NLRI --> P2[203.0.113.0/24]
```

- Network Layer Reachability Information (NLRI): Lists the IP prefixes being advertised.
- Path Attributes: Contains metadata for the prefixes (e.g., AS_PATH, NEXT_HOP, LOCAL_PREF) used for path selection.
- Withdrawn Routes: Lists prefixes that are no longer reachable. [2, 6, 10]



It's important to not that a single UPDATE message can advertise multiple prefixes but they all share the same path attributes.

2.4 3. NOTIFICATION (Type 3)

A NOTIFICATION message is sent whenever BGP detects an error. Upon sending or receiving this message, the BGP session and underlying TCP connection are immediately closed. It contains: [4, 5, 11]

- Error Code: Major error category (e.g., Hold Timer expired, Message Header error).
- Error Subcode: Specific details about the error. [3, 6]

2.5 4. KEEPALIVE (Type 4) [3]

KEEPALIVE messages are sent periodically (default 60 seconds) to ensure the peer is still active. They consist only of the 19-byte header and carry no data. If no Keepalive or Update messages are received within the negotiated Hold Time, the session terminates. [1, 2, 10, 11, 12]

2.6 5. ROUTE-REFRESH (Type 5) [13]

Defined in RFC 2918, the ROUTE-REFRESH message allows a router to request its peer to resend its full routing table for a specific address family without resetting the session. This is commonly used to apply new inbound routing policies non-disruptively. [14, 15, 16]

[1] https://support.huawei.com [2] https://networklessons.com [3] https://www.iana.org [4] https://ipcisco.com [5] https://support.huawei.com [6] https://flylib.com [7] https://www.vervecopilot.com [8] https://www.naukri.com [9] https://networklessons.com [10] https://www.scribd.com [11] https://stormwall.network [12] https://support.huawei.com [13] https://www.ciscopress.com [14] https://pike.lysator.liu.se [15] https://support.huawei.com [16] https://documentation.nokia.com

3 Path Attributes and BGP Best Path Selection

3.1 Path Attributes

Path attributes are the "tags" or metadata attached to NLRI (prefixes) within a BGP UPDATE message. BGP uses these attributes to determine the best path to a destination and to detect routing loops.

```
graph TD
    PA[Path Attributes] --> WK[Well-Known]
    PA --> OP[Optional]

    WK --> WKM[Mandatory]
    WK --> WKD[Discretionary]

    OP --> OPT[Transitive]
    OP --> OPN[Non-Transitive]

    WKM --- AS_PATH
    WKM --- NEXT_HOP
    WKM --- ORIGIN

    WKD --- LOCAL_PREF

    OPT --- COMMUNITY

    OPN --- MED
```

They are categorized into four types:

- Well-known Mandatory (must be supported and present),
- Well-known Discretionary (must be supported, may be present),
- Optional Transitive (passed along even if not understood),
- Optional Non-transitive (discarded if not understood).

Attributes are communicated through the UPDATE message and applied by each bgp speaker locally to determine the best paths to each destination.

Below follows a list of the most used and important attributes.

3.1.1 1. AS_PATH (Well-known Mandatory)

- What it is: A list of all Autonomous System numbers a route has passed through.
- Role: It is used for loop prevention (if a router sees its own AS in the list, it drops the update) and path selection (shorter AS_PATHs are generally preferred).

3.1.2 2. NEXT_HOP (Well-known Mandatory)

- What it is: The IP address of the router used to reach the advertised prefixes.
- Role: For eBGP, this is typically the IP of the neighbor; for iBGP, it usually remains the IP of the original eBGP exit point unless "next-hop-self" is configured.

3.1.3 3. ORIGIN (Well-known Mandatory)

- What it is: Defines where the route came from.
- Values:
- IGP (0): Interior Gateway Protocol (e.g., via a network command).
  - EGP (1): Exterior Gateway Protocol (historic, rarely seen).
  - Incomplete (?): Unknown origin (e.g., redistributed from another protocol).

3.1.4 4. LOCAL_PREF (Well-known Discretionary)

- What it is: A value telling routers within an AS which path is preferred to exit the AS.
- Role: Higher is better. It is not passed between different Autonomous Systems; it stays internal to your network.

3.1.5 5. MULTI_EXIT_DISC / MED (Optional Non-transitive)

- What it is: A hint sent to an external neighbor about the preferred entry point into your AS.
- Role: Lower is better. Unlike Local Pref, this is used to influence inbound traffic from a neighbor.

3.1.6 6. COMMUNITY (Optional Transitive)

- What it is: A numerical tag used to group routes for administrative purposes.
- Role: Used to trigger specific routing policies (e.g., "don't advertise this to Peer X" or "set Local Pref to 200").

3.2 BGP Best Path Selection Algorithm

BGP routers often receive multiple paths to the same destination. To pick the single "best" path to install in the routing table, BGP follows a strict, step-by-step tie-breaker list. The algorithm stops at the first step where one path is clearly superior to the others.

```
flowchart TD
    START([Start])
 --> W{Weight?}
    W -->|Tie| LP{Local Pref?}
    LP -->|Tie| LO{Locally Originated?}
    LO -->|Tie| AS{AS Path Length?}
    AS -->|Tie| OC{Origin Code?}
    OC -->|Tie| MED{MED?}
    MED -->|Tie| NT{Neighbor Type eBGP > iBGP?}
    NT -->|Tie| MET{IGP Metric to NH?}
    MET -->|Tie| TB[Tie-Breakers]

    W -->|Highest| WIN([WINNER])
    LP -->|Highest| WIN
    LO -->|Yes| WIN
    AS -->|Shortest| WIN
    OC -->|Lowest| WIN
    MED -->|Lowest| WIN
    NT -->|eBGP| WIN
    MET -->|Lowest| WIN
```

3.2.1 The BGP Best Path Selection Hierarchy

1. Weight (Cisco specific): Highest weight is preferred. This is local to the router and not advertised to any peers.
2. Local Preference: Highest Local Preference is preferred. This is used to choose the exit point from your local AS.
3. Locally Originated: Prefer paths that were locally originated (via network or aggregate commands) over those learned from a peer.
4. AS_PATH Length: Shortest AS_PATH is preferred. Each AS number the route passes through adds "length."
5. Origin Type: Lowest origin code is preferred: IGP < EGP < Incomplete.
6. Multi-Exit Discriminator (MED): Lowest MED is preferred. This is typically used to influence how neighbors enter your AS.
7. Neighbor Type: Prefer eBGP paths over iBGP paths.
8. IGP Metric to Next Hop: Prefer the path with the lowest internal gateway protocol (OSPF/IS-IS/EIGRP) metric to the BGP NEXT_HOP.

3.2.2 Tie-Breakers (If everything above is equal)

If the paths are still identical after the main checks, BGP uses these final "last resort" steps: 9. Oldest Path: Prefer the path that was received first (most stable). 10. Lowest Router ID: Prefer the path coming from the neighbor with the lowest BGP Router ID. 11. Lowest Neighbor IP: If multiple sessions exist with the same neighbor, prefer the lowest neighbor IP address.

3.2.3 Pro-Tip: Memory Hook

A common mnemonic to remember the main steps is **W**e **L**ove **O**range **A**s **O**ur **M**other **E**ats **I**ce-cream:

- Weight
- Local Pref
- Originate
- AS_Path
- Origin Code
- MED
- External (eBGP)
- IGP metric

4 Announcing My Routes to the World Using BGP

In previous chapters , we looked at how to establish sessions and how BGP selects the best path. Now, we will focus on how to actually tell the world about the prefixes we own. BGP is a very conservative protocol; it will not announce anything unless specifically told to do so, and even then, certain conditions must be met.

4.1 Multiprotocol BGP and Address Families

Modern BGP is "Multiprotocol" (MP-BGP). This means it can carry routing information for different types of addresses (IPv4, IPv6, VPNs, etc.) over the same session. To manage this, we use **Address Families**.

In the past, BGP configuration was flat. Today, we define the neighbor globally but activate specific capabilities and prefixes within an `address-family` block.

```
router bgp 65001
 neighbor 2001:db8:acad::2 remote-as 65002
 !
 address-family ipv4 unicast
  neighbor 2001:db8:acad::2 activate
  network 192.0.2.0 mask 255.255.255.0
 exit-address-family
 !
 address-family ipv6 unicast
  neighbor 2001:db8:acad::2 activate
  network 2001:db8:beef::/48
 exit-address-family
```

Note: You can carry IPv4 prefixes over an IPv6 peering session (and vice versa), which is very common in modern dual-stack networks.

4.2 The `network` Command

The `network` command tells BGP: *"If you have this exact prefix in your routing table, announce it to my peers."*

4.2.1 The "Exact Match" Rule

For the `network` command to work, the prefix and its mask must match an entry in the IP routing table (RIB) **exactly**.

```
graph TD
    A[network 192.0.2.0/24] --> B{Match in RIB?}
    B -- No --> C[No Announcement]
    B -- Yes --> D[Advertise to Peers]

    subgraph RIB
    E[192.0.2.0/24 Connected]
    F[192.0.2.0/24 Static]
    G[192.0.2.0/25 OSPF]
    end

    E -.-> B
    F -.-> B
    G -.->|No Match| B
```

IPv4 Example:

```
address-family ipv4 unicast
 network 192.0.2.0 mask 255.255.255.0
```

IPv6 Example:

```
address-family ipv6 unicast
 network 2001:db8:beef::/48
```

4.3 Anchoring Routes: The "Pull-up" Route

To satisfy the "Exact Match" rule when you don't have a specific interface for a block, we use a **pull-up** route pointing to a discard interface.

```
graph LR
    A[BGP Process]
 -->|Scans RIB| B[Static Route: 192.0.2.0/24 -> Null0]
    B -->|Match Found| C[Announce Prefix to Internet]
    D[Traffic from Internet] -->|Destination: 192.0.2.5| E[Router]
    E -->|Matches Anchor| B
    B -->|Discard| F((Null0))
```

4.3.1 Cisco IOS Example

```
ip route 192.0.2.0 255.255.255.0 Null0
ipv6 route 2001:db8:beef::/48 Null0
```

4.3.2 FRR Note: The Blackhole Route

In FRR, use the `blackhole` keyword:

```
ip route 192.0.2.0/24 blackhole
ipv6 route 2001:db8:beef::/48 blackhole
```

4.4 Historical Context: BGP Synchronization

The Rule: Do not advertise an iBGP route to an eBGP peer unless the IGP also knows about it. **Status:** Usually disabled (`no synchronization`) because we now use full iBGP meshes or MPLS.

4.5 Redistributing IGPs into BGP

Redistribution pulls routes from OSPF, IS-IS, or Connected interfaces into BGP automatically. This must be done inside the correct address family.

```
address-family ipv6 unicast
 redistribute ospf 1
```

4.5.1 Risks: Instability and Leaks

1. **Leaking Internal Routes:** You might announce private `/32` or `/128` loopbacks.
2. **Route Flapping:** Internal link issues can cause global BGP instability.

4.5.2 Control with Route-Maps

```
ipv6 prefix-list MY-V6-BLOCK permit 2001:db8:beef::/48
!
route-map OSPF-TO-BGP-V6 permit 10
 match ipv6 address prefix-list MY-V6-BLOCK
!
router bgp 65001
 address-family ipv6 unicast
  redistribute ospf 1 route-map OSPF-TO-BGP-V6
```

4.6 Influencing Inbound Traffic: The MED

The **Multi-Exit Discriminator (MED)** is a metric sent to eBGP peers to "suggest" an entry point. **Lower is preferred.**

```
graph RL
    subgraph My AS 65001
    R1[Router A]
    R2[Router B]
    end

    subgraph ISP AS 65002
    ISP[ISP Router]
    end

    R1 -- "MED 100 (Preferred)" --> ISP
    R2 -- "MED 200" --> ISP

    ISP -.->|Traffic In| R1
```

```
route-map SET-MED-LOW permit 10
 set metric 100

router bgp 65001
 address-family ipv4 unicast
  neighbor 10.0.0.2 route-map SET-MED-LOW out
```

4.7 Troubleshooting and Verification

When things don't work, you need to check the RIB, the BGP Table, and the Neighbor state.

4.7.1 1. Verification Commands

- `show ip bgp` (or `show bgp ipv6 unicast`): Shows the BGP table. Look for the `*` (valid) and `>` (best) symbols.
- `show ip route`: Checks the main routing table to see if your "Exact Match" or "Anchor" exists.
- `show ip bgp neighbors <IP> advertised-routes`: Checks exactly what you are sending to a peer.
- `show ip bgp neighbors <IP> routes`: Checks what you are receiving from a peer.

4.7.2 2. Clearing the Session (The "Hammer")

If you change a policy (like a route-map), BGP doesn't always apply it immediately to existing routes. You may need to reset the session.

- **Hard Reset:** `clear ip bgp *`
  - **Effect:** Drops the TCP session and clears the BGP table.
  - **Warning:** Causes a complete traffic outage for a few seconds. **Never do this in production unless necessary.**
- **Soft Reset (Dynamic):** `clear ip bgp * soft out` (or `in`)
  - **Effect:** Asks the peer to re-send updates (Route Refresh) without dropping the session. No traffic loss.

4.7.3 3. Soft Reconfiguration Inbound

If a neighbor doesn't support "Route Refresh," you can use `soft-reconfiguration inbound`.

```
neighbor 10.0.0.2 soft-reconfiguration inbound
```

- **Advantage:** Allows you to see the "raw" routes received from a neighbor before any of your filters are applied (`show ip bgp neighbors 10.0.0.2 received-routes`).
- **Disadvantage:** Memory intensive. The router has to store a second copy of every route the neighbor sends, which can be millions of prefixes on the full Internet table.

4.8 Summary Checklist

☐ Are the correct **Address Families** activated?
☐ Does the prefix exist in the RIB? (Exact Match/Anchor)
☐ If using redistribution, is it filtered?
☐ Have you cleared the session (soft) after applying new policies?

5 Chapter 06: iBGP Mechanics and Internal Propagation

In previous chapters , we focused mostly on eBGP (External BGP) to connect different Autonomous Systems. However, most networks have more than one BGP router. How these internal routers share BGP information is the realm of **Internal BGP (iBGP)**.

iBGP is not just "BGP between friends"; it has very specific rules that differ from eBGP to prevent loops and ensure stability.

5.1 1. Peering on Loopbacks

In eBGP, we usually peer using the IP addresses of the physical link connecting the two routers. In iBGP, we almost always peer using **Loopback interfaces**.

5.1.1 Why?

If a physical link between two internal routers fails, but there is another path through the network (OSPF/IS-IS), the BGP session should stay up. Since a Loopback interface never goes down unless the router itself fails, it provides a stable anchor for the TCP session.

The `update-source` command: When peering with a Loopback, you must tell BGP to use that Loopback's IP as the source of its BGP packets, otherwise, the neighbor will reject the connection.

```
router bgp 65001
 neighbor 1.1.1.1 remote-as 65001
 neighbor 1.1.1.1 update-source Loopback0
```

5.2 2. The iBGP Split-Horizon Rule

This is the most important rule in iBGP:

> **"A route learned from an iBGP peer will never be advertised to another iBGP peer."**

```
graph LR
    subgraph AS 65001
    R1[R1 Border] -- iBGP --> R2[R2 Internal]
    R2 -- "iBGP (Blocked)" --> R3[R3 Internal]
    end
    ISP(ISP) -- eBGP Update --> R1

    style R2 fill:#f96,stroke:#333
    linkStyle 1 stroke:#ff0000,stroke-width:2px,stroke-dasharray: 5 5;
```

5.2.1 The Reason

In eBGP, we use the `AS_PATH` to detect loops. If we see our own AS in the path, we reject the route. However, inside an AS, the `AS_PATH` doesn't change. Without this Split-Horizon rule, a route could loop forever inside the AS.

5.2.2 The Consequence: Full Mesh

Because of Split-Horizon, every iBGP router must have a direct session with **every other** iBGP router in the AS to ensure everyone sees all routes. * 3 routers = 3 sessions * 10 routers = 45 sessions * 100 routers = 4,950 sessions! (This doesn't scale well).

5.3 3. The Next-Hop Behavior (The #1 iBGP Issue)

By default, BGP **does not change the Next-Hop attribute** when sending a route to an iBGP peer.

```
graph LR
    ISP[ISP Router 10.0.0.1]
 -- eBGP Update --> R1[R1 Border]
    subgraph AS 65001
    R1 -- "iBGP (Next-Hop: 10.0.0.1)" --> R2[R2 Internal]
    end

    R2 -- "Can I reach 10.0.0.1?" --> IGP{IGP Table}
    IGP -- No --> FAIL[Route Inaccessible]
```

5.3.1 The Problem

If Router A (eBGP) receives a route from an ISP and sends it to Router B (iBGP), the Next-Hop remains the IP of the ISP's router. If Router B doesn't have a route to that ISP's IP in its IGP (OSPF), it will mark the BGP route as **"Invalid"** or **"Inaccessible"**.

5.3.2 The Solution: `next-hop-self`

We tell the border router to change the Next-Hop to its own Loopback IP before sending it to internal peers.

```
router bgp 65001
 address-family ipv4 unicast
  neighbor 1.1.1.1 next-hop-self
```

5.4 4. TTL Differences

- **eBGP:** Packets are sent with a Time-to-Live (TTL) of **1**. This assumes the neighbor is directly connected. (Peering on eBGP loopbacks requires `ebgp-multihop`).
- **iBGP:** Packets are sent with a TTL of **255**. iBGP peers can be many hops away as long as there is IGP reachability.

5.5 5. Scaling iBGP: Route Reflectors

Since a Full Mesh is impossible in large networks, we use **Route Reflectors (RR)** to "break" the Split-Horizon rule safely.

```
graph TD
    RR[Route Reflector]
    C1[Client 1]
    C2[Client 2]
    NC[Non-Client]

    RR <-->|iBGP| C1
    RR <-->|iBGP| C2
    RR <-->|iBGP| NC

    C1 -.->|Update| RR
    RR -.->|Reflects| C2
    RR -.->|Reflects| NC
```

A Route Reflector is a router allowed to re-advertise iBGP routes to other iBGP peers (its "clients"). To prevent loops, it adds two special attributes: 1. **ORIGINATOR_ID:** The Router ID of the router that first sent the route. 2. **CLUSTER_LIST:** A list of RRs the route has passed through.

5.5.1 RR Configuration Example

Only the Route Reflector needs special configuration:

```
router bgp 65001
 neighbor 2.2.2.2 remote-as 65001
 address-family ipv4 unicast
  neighbor 2.2.2.2 route-reflector-client
```

5.6 Summary Comparison

Feature	eBGP	iBGP
AS Number	Different	Same
TTL	1 (default)	255
Next-Hop	Changes to self	Does not change (by default)
Propagation	Re-advertises everything	Does not re-advertise iBGP to iBGP
Loop Prevention	AS_PATH	Split-Horizon

5.7 Troubleshooting Checklist

- ☐ Is the `update-source` set to the Loopback?
- ☐ Does the IGP (OSPF/Static) have a route to the neighbor's Loopback?
- ☐ Is `next-hop-self` configured on the border routers?
- ☐ If not using a full mesh, is a Route Reflector configured?

6 Accepting Routes from Others

In the previous chapters, we focused on how to establish BGP sessions and how to tell the world about *our* networks. Now, we turn the tables: how do we safely and effectively receive routing information from the rest of the world?

When you connect to an ISP or a peer, they will send you BGP updates. Accepting these updates blindly is a recipe for disaster. In this chapter, we will learn how to build a **Routing Policy** to control exactly what enters our network.

6.1 1. Defining Routing Policy

A **Routing Policy** is a set of rules that determines how a router handles routing information. In BGP, policies are typically applied at the "edge" of your network—where you interface with other Autonomous Systems.

6.1.1 The Inbound Pipeline

When a BGP update arrives from a neighbor, it doesn't immediately enter your BGP table (Loc-RIB). It passes through a series of filters first.

```
graph LR
    A[Neighbor Update] --> B{Inbound Filter}
    B -- Deny --> C[Discard]
    B -- Permit --> D[BGP Table - Loc-RIB]
    D --> E{Best Path Selection}
    E --> F[IP Routing Table - RIB]
```

By applying a policy **inbound**, you control: 1. **Memory Usage:** By rejecting prefixes you don't need. 2. **Traffic Flow:** By modifying attributes (like Local Preference) to prefer one provider over another. 3. **Security:** By rejecting invalid or malicious routes.

6.2 2. The Toolset: Prefix-Lists, AS-PATH Lists, and Route-Maps

To build a policy, we use three primary tools in FRR.

6.2.1 Prefix-Lists

Prefix-lists match routes based on the **network address** and the **mask length**. They are much more efficient than standard Access Control Lists (ACLs) for routing.

- **Logic:** `ip prefix-list NAME [permit|deny] PREFIX/LEN [ge MIN-MASK] [le MAX-MASK]`
- **Example:** `ip prefix-list ONLY-SMALL-BLOCKS permit 0.0.0.0/0 ge 24 le 32` (Matches any prefix with a mask between /24 and /32).

6.2.2 AS-PATH Access-Lists

These match routes based on the **AS_PATH** attribute using Regular Expressions (Regex).

- ^ : Start of the path.
- **$** : End of the path.
- **_** : Any delimiter (space, comma, start/end).
- **.*** : Anything.

Common Regex Patterns: | Pattern | Meaning | | :— | :— | | `^$` | Routes originated by the local AS (Internal). | | `^65001$` | Routes originated by AS 65001 and directly connected. | | `_65001_` | Any route that has passed through AS 65001. | | `^65001 .*` | Routes originated by AS 65001 or any of its customers. |

6.2.3 Route-Maps

The "Swiss Army Knife" of routing. Route-maps use `match` statements to find routes and `set` statements to change their attributes.

```
route-map MY-POLICY permit 10
 match ip address prefix-list MY-PREFIXES
 match as-path MY-AS-PATHS
 set local-preference 200
```

6.3 3. Hands-on Scenario 1: The "Default Route Only" Filter

Imagine you have a low-end router with limited memory. You only want a **Default Route** (0.0.0.0/0) from your ISP to reach the Internet. You don't want the 900,000+ prefixes of the full Internet table.

6.3.1 Configuration

1. **Define the Prefix-List:**

```
ip prefix-list PL-DEFAULT-ONLY permit 0.0.0.0/0
```

2. **Create the Route-Map:**

```
route-map RM-DEFAULT-ONLY permit 10
 match ip address prefix-list PL-DEFAULT-ONLY
```

3. **Apply to the Neighbor (Inbound):**

```
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.0.13.3 route-map RM-DEFAULT-ONLY in
 exit-address-family
```

6.4 4. Hands-on Scenario 2: Filtering by Origin (AS-PATH Regex)

Sometimes you want to accept routes from a neighbor, but **only** the routes that they actually own (originate). You don't want to accept routes they learned from their other peers (which could turn you into an accidental transit provider!).

6.4.1 The Task

Accept only routes where the AS_PATH consists of exactly one AS: **65001**.

6.4.2 Configuration

1. **Define the AS-PATH List:**

```
ip as-path access-list AS-OWNED-ONLY permit ^65001$
```

2. **Create the Route-Map:**

```
route-map RM-OWNED-ONLY permit 10
 match as-path AS-OWNED-ONLY
```

3. **Apply Inbound:**

```
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.0.13.3 route-map RM-OWNED-ONLY in
 exit-address-family
```

6.5 5. Hands-on Scenario 3: Defensive Hygiene & Prefix Limits

6.5.1 The "Bogon" Filter

"Bogons" are IP spaces that should never appear in the public BGP table (e.g., private ranges, reserved space). Accepting them can lead to routing loops or security holes.

```
ip prefix-list PL-BOGONS deny 10.0.0.0/8 le 32
ip prefix-list PL-BOGONS deny 172.16.0.0/12 le 32
ip prefix-list PL-BOGONS deny 192.168.0.0/16 le 32
ip prefix-list PL-BOGONS permit 0.0.0.0/0 le 32
```

6.5.2 Prefix Limits

What if your neighbor accidentally sends you 1,000,000 routes instead of the expected 10? To protect your router, you should set a **Maximum Prefix** limit.

```
router bgp 65000
 neighbor 10.0.13.3 maximum-prefix 1000
```

If the neighbor sends more than 1,000 prefixes, the BGP session will immediately **shutdown** to protect your memory. It will remain in the `Idle (PfxCt)` state until you manually reset it.

6.6 6. Verification and "Soft Resets"

How do you know if your policy is working?

6.6.1 The "Received" vs. "Accepted" distinction

- **Received:** Every route the neighbor sent (requires `soft-reconfiguration inbound`).
- **Accepted:** The routes that survived your filters and are in your BGP table.

Commands: * `show ip bgp neighbor 10.0.13.3 received-routes` (What they sent). * `show ip bgp neighbor 10.0.13.3 routes` (What you kept).

6.6.2 Applying Policy Changes

If you change a route-map, BGP doesn't automatically re-process existing routes. You must trigger a "Soft Reset". Unlike a hard reset, this **does not drop the TCP session**.

```
clear ip bgp 10.0.13.3 soft in
```

This tells the router: *"Ask the neighbor to re-send their updates so I can run them through my new filters."*

7 Traffic Engineering with BGP

In the previous chapter, we learned how to safely accept routes from others. Now, we move to the next level of BGP operations: **Traffic Engineering (TE)**.

Traffic Engineering is the process of steering traffic across your network to achieve specific goals, such as: * Preferring a cheaper provider over an expensive one. * Using a high-bandwidth link for heavy traffic. * Ensuring a backup link remains idle unless the primary fails.

7.1 1. The Two Directions of Traffic

When performing TE, you must always think in two directions: 1. **Outbound Traffic (Traffic Leaving Your Network):** How *your* users reach the Internet. This is easy to control because you make the decision locally. 2. **Inbound Traffic (Traffic Entering Your Network):** How the *Internet* reaches your users. This is harder to control because you are trying to influence decisions made by routers you don't own.

> **Crucial Rule:** Traffic follows the **opposite** path of the BGP advertisements. If you want traffic to come *in* via Provider A, you must make your BGP advertisements to Provider A look more attractive than those to Provider B.

7.2 2. Influencing Outbound Traffic

To choose which "exit" your traffic takes, the primary tool is **Local Preference**.

7.2.1 Local Preference (Local-Pref)

- **Scope:** Entire Autonomous System (iBGP).
- **Preference:** Higher is better (Default is 100).
- **Use Case:** If you have two ISPs, and you want to prefer ISP-A for all traffic.

7.2.1.1 Hands-on: Preferring an ISP with Local-Pref

Imagine R1 is connected to ISP-A and R2 is connected to ISP-B. We want to prefer ISP-A.

```
! On R1 (Connected to ISP-A)
route-map RM-PREFER-ISP-A permit 10
 set local-preference 200
!
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.0.1.1 route-map RM-PREFER-ISP-A in
 exit-address-family
```

7.2.2 Weight (FRR/Cisco Specific)

- **Scope:** Local router only (does not pass to iBGP peers).
- **Preference:** Higher is better (Default 0).
- **Use Case:** Quickly choosing an exit point on a single router with multiple links.

7.3 3. Influencing Inbound Traffic

This is the "Art" of BGP. You are trying to convince the rest of the world to prefer one of your links over another.

7.3.1 AS-PATH Prepending

BGP prefers shorter AS_PATHs. To make a link *less* attractive, you can artificially lengthen the path by repeating your own ASN multiple times.

7.3.1.1 Hands-on: Making ISP-B a Backup Link

We want the world to use ISP-A to reach us. We will "prepend" our ASN (65000) three times towards ISP-B.

```
! On the router connected to ISP-B
route-map RM-AS-PREPEND permit 10
 set as-path prepend 65000 65000 65000
!
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.0.2.1 route-map RM-AS-PREPEND out
 exit-address-family
```

7.3.2 The "More-Specific" Trick

Traffic always follows the **Longest Match**. If you announce `192.0.2.0/24` to ISP-A and `192.0.2.0/25` + `192.0.2.128/25` to ISP-B, all traffic for that range will go to ISP-B, regardless of any other BGP attributes. * *Warning:* This contributes to global routing table bloat and should be used sparingly.

7.4 4. BGP Communities: The "Remote Control"

A **Community** is a 32-bit tag (usually written as `ASN:VALUE`) added to a route. It doesn't change path selection by itself, but it can trigger actions on other routers.

7.4.1 Standard Communities

- **no-export (65535:65281):** "Do not announce this route outside of my AS."
- **no-advertise (65535:65282):** "Do not announce this route to *any* peer (iBGP or eBGP)."

7.4.1.1 Hands-on: Using No-Export

You want to announce a prefix to your ISP so they can reach you, but you don't want them to pass that route to their other customers or providers.

```
route-map RM-INTERNAL-ONLY permit 10
 set community no-export
!
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.0.1.1 route-map RM-INTERNAL-ONLY out
  neighbor 10.0.1.1 send-community
 exit-address-family
```

Note: You must explicitly use ***`send-community`*** *in FRR, or the tags will be stripped before the update is sent!*

7.5 5. Summary: The TE Hierarchy

When you have multiple attributes set, BGP follows a strict hierarchy (The Best Path Selection Algorithm).

Attribute	Direction Controlled	Scope	Preference
Weight	Outbound	Local Router	Higher
Local-Pref	Outbound	Entire AS	Higher
AS-PATH	Inbound	Global	Shorter
MED	Inbound	Between two ASs	Lower
Communities	Both	Variable	N/A

7.5.1 Pro-Tip: The "Soft Reset"

Remember, whenever you change a `set` command in a route-map, you must trigger a reset to see the effects: `clear ip bgp <neighbor> soft out` (for outbound policy) `clear ip bgp <neighbor> soft in` (for inbound policy)